Web Security Concepts

Session Cookies & X-Forwarded-Host Header

1. Session Cookies

What is a Session Cookie?

A **session cookie** is a small piece of data stored in the browser that identifies a user's session on a web server. Because HTTP is stateless (each request is independent), session cookies let the server recognise the same user across multiple requests — for example, keeping you logged in as you browse different pages.

How it works — step by step:

Step	Actor	Action
1	Browser	Sends login credentials (username + password) to the server.
2	Server	Validates credentials, creates a unique session ID, stores session data.
3	Server	Sends back a Set-Cookie header containing the session ID.
4	Browser	Stores the cookie and attaches it to every subsequent request automatically.
5	Server	Reads the cookie, looks up the session, and knows who the user is.

HTTP Example

Server response after successful login — the browser is told to store the cookie:

```
HTTP/1.1 200 OK Set-Cookie: sessionId=abc123xyz789; Path=/; HttpOnly; Secure;
SameSite=Strict Content-Type: text/html
```

Browser's subsequent request — the cookie is sent back automatically:

```
GET /dashboard HTTP/1.1 Host: example.com Cookie: sessionId=abc123xyz789
```

Security Flags Explained

Flag	Meaning
HttpOnly	Cookie cannot be read by JavaScript (blocks XSS theft).
Secure	Cookie is sent only over HTTPS connections.
SameSite=Strict	Cookie is not sent on cross-site requests (blocks CSRF).

SameSite=Lax	Cookie sent on top-level navigations but not sub-requests.
Expires / Max-Age	Controls when the cookie is deleted by the browser.

Common Attacks on Session Cookies

Session Hijacking: An attacker steals the session cookie (e.g., via XSS or network sniffing) and replays it to impersonate the victim. Mitigate with HttpOnly, Secure, and short expiry.

Session Fixation: Attacker forces the victim to use a known session ID. Mitigate by regenerating the session ID after login.

Cross-Site Request Forgery (CSRF): A malicious page tricks the browser into sending the cookie to a target site. Mitigate with SameSite=Strict and CSRF tokens.

■ Never store sensitive data (passwords, credit-card numbers) inside the cookie itself — store only a random session ID and keep the real data server-side.

2. X-Forwarded-Host Header

What is X-Forwarded-Host?

The **X-Forwarded-Host** (XFH) header is an HTTP request header added by a **proxy** or **load balancer** to tell the origin server the original *Host* header value the client used. When a request passes through intermediary servers, the Host header may change; XFH preserves the original hostname so the backend can generate correct URLs, redirects, and links.

Normal (legitimate) usage — behind a reverse proxy:

```
# Client contacts the load balancer at api.example.com GET /users HTTP/1.1 Host:
load-balancer.internal X-Forwarded-Host: api.example.com X-Forwarded-For: 203.0.113.55
# Backend sees X-Forwarded-Host and uses api.example.com for any # absolute URL it
generates (e.g. password-reset links).
```

Architecture diagram (text):

```
[Client] ---HTTPS---> [Load Balancer / CDN] ---HTTP---> [Origin Server] Host:
api.example.com adds X-Forwarded-Host reads XFH header
```

Security Risk: Host Header Injection

If the back-end server **blindly trusts** the X-Forwarded-Host header and uses it to construct URLs (e.g., password-reset links, OAuth redirect URIs), an attacker can inject an arbitrary hostname and redirect victims to a malicious site.

Attack example — password reset poisoning:

```
# Attacker sends this request to the /forgot-password endpoint: POST /forgot-password
HTTP/1.1 Host: vulnerable-app.com X-Forwarded-Host: attacker.com # If the server builds
the reset link from X-Forwarded-Host: # Victim receives email with link:
https://attacker.com/reset?token=ABC123 # Victim clicks, token is sent to attacker —
account taken over!
```

Other attack vectors:

Web Cache Poisoning: Inject a malicious host into a cached response so other users receive it.

Open Redirect: Forge the host to redirect users to phishing pages.

SSRF via Host: In some configurations, trick the server into making requests to internal services.

How to Defend Against XFH Injection

Defence	Details
Whitelist allowed hosts	Only accept known, expected values of X-Forwarded-Host. Reject or ignore others.
Use a hard-coded base URL	Configure the application's base URL explicitly (env var / config file). Never derive it from headers.

Restrict at the proxy	Strip or overwrite X-Forwarded-Host at the load balancer before forwarding to the backend.
Validate in middleware	In Django: ALLOWED_HOSTS. In Express: trust proxy setting + host validation middleware.

✓ Best practice: treat X-Forwarded-Host as untrusted user input unless your infrastructure explicitly sets and controls it.

3. Quick Comparison

	Session Cookie	X-Forwarded-Host
Type	Response header (server→browser) stored as browser cookie	Request header (proxy→server)
Purpose	Maintain user state / identity across requests	Preserve original hostname through proxies
Set by	Web server (Set-Cookie header)	Reverse proxy / load balancer
Read by	Browser (sent automatically) & server	Origin / backend server
Key risk	Theft, fixation, CSRF	Host header injection, cache poisoning
Mitigation	HttpOnly, Secure, SameSite flags	Whitelist, hard-coded base URL, proxy stripping